

Magic Pro Architecture Report

Browser-based DAW architecture, current implementation map, integration boundaries, and launch-critical system design

Workspace: C:\personal\daw

Generated: 2026-05-21 19:55

Audience: Engineering, product, launch readiness

1. Executive Architecture Summary

Magic Pro is implemented as a Next.js 14 application with a desktop-class DAW interface, a Zustand-backed project state model, Prisma/Postgres server persistence, IndexedDB-oriented filesystem services, and multiple browser audio subsystems built around Web Audio, AudioWorklets, scheduler workers, synthetic instruments, sample playback, and experimental DSP/WASM modules.

Architecturally, the project is not a small app with one clean vertical slice. It is a broad DAW workbench with several partially overlapping generations of engine code. The main runtime path used by the current studio page is `app/project/[projectId]/page.tsx -> store/projectStore.ts -> engine/AudioEngineAdapter.ts -> engine/audioEngine` modules.

Architecture state: The repository contains both production-facing systems and scaffold/prototype systems. Any architecture diagram must distinguish the current wired path from aspirational or parallel systems.

Layer	Primary implementation	Role
App shell	<code>app/</code> , <code>components/</code>	Next.js App Router pages, DAW workspace, panels, dialogs, editors.
State	<code>store/projectStore.ts</code> ; <code>stores/</code>	Project, transport, clips, mixer, UI visibility, preferences, persistence actions.
Engine adapter	<code>engine/AudioEngineAdapter.ts</code>	Compatibility facade between UI/store and modular audio engine.
Audio V2	<code>engine/audioEngine/</code>	AudioContext, scheduler, routing, recording, bounce, meters, channel strips.
DSP/WASM	<code>engine/audio/</code> , <code>engine/dsp/</code> , <code>rust/</code> , <code>wasm/</code>	Graph compiler, runtime scaffolds, WASM plugins/kernels, worklet direction.
Persistence	<code>app/api/project/*</code> , <code>prisma/</code> , <code>engine/filesystem/</code>	Server database persistence, local project serialization, IndexedDB concepts.
Assets	<code>public/sound_sample/</code> , <code>lib/libraryData.ts</code>	Bundled piano/guitar samples, library preset metadata, remote loop references.

2. Top-Level Runtime Topology

Browser
Next.js App Router
v
<code>app/page.tsx -> app/dashboard/page.tsx -> app/project/[projectId]/page.tsx</code>
React components
v
<code>TransportBar / TrackList / Timeline / Mixer / PianoRoll / LibraryPanel / Browsers</code>
Zustand actions and selectors
v
<code>store/projectStore.ts</code>
Audio compatibility facade

```

v
engine/AudioEngineAdapter.ts
|
+--> engine/audioEngine/audioContext.ts
+--> engine/audioEngine/scheduler.ts
+--> engine/audioEngine/routingEngine.ts
+--> engine/audioRecording/recorder.ts
+--> engine/instruments/*
+--> engine/audioEngine/bounceEngine.ts

Server/API side
|
+--> app/api/auth/[...nextauth]/route.ts
+--> app/api/project/save/route.ts
+--> app/api/project/[id]/route.ts
+--> lib/prisma.ts -> prisma/schema.prisma -> PostgreSQL

```

3. Application Routing Architecture

Route / file	Runtime type	Responsibility	Current behavior
app/page.tsx	Server component	Root entry	Redirects immediately to /dashboard.
app/dashboard/page.tsx	Client component	Authenticated project landing	Checks NextAuth session, opens ProjectChooser, links demo projects.
app/login/page.tsx	Client component	Sign-in UI	Demo credentials flow; no real signup/account surface.
app/project/[projectId]/page.tsx	Client component	Main DAW studio	Mounts transport, timeline, mixer, panels, dialogs, virtual keyboard, and useAudioPlayer.
app/debug-audio/page.tsx	Client/debug	Audio diagnostics surface	Useful for local diagnosis, not a product route.
app/api/auth/[...nextauth]/route.ts	API route	NextAuth credentials provider	Any authorize call returns a demo user.
app/api/project/save/route.ts	API route	Project upsert	Writes Project root and recreates track graph in Prisma.
app/api/project/[id]/route.ts	API route	Project load	Reads project with nested relations and stateJson.

4. Frontend Composition

The main studio page is a composed DAW workspace. It imports the major panels directly rather than routing into nested pages. This makes the studio page the central composition root.

Component group	Files	Function
Transport and global chrome	TransportBar, AppMenuBar, Toolbar, ViewControlBar, GlobalKeyHandler	Top/bottom controls, keyboard shortcuts, modal toggles, transport commands.
Arrangement	TrackList, Timeline, GlobalTracks, LiveLoopsGrid	Track headers, clips/regions, global tempo/signature lanes, live loops surface.
Editing	PianoRoll, StepInputKeyboard, AudioTrackEditor, SelectionBasedProcessing	MIDI note editing, step input, audio region processing surfaces.
Mixing	Mixer, components/mixer/*, PluginEditorWindow, SmartControls	Channel strips, faders, meters, sends, plugin UI shells.
Assets	LibraryPanel, LoopBrowser, Browsers	Preset selection, loops, imports, project files.
Project operations	ProjectChooser, ProjectManager, SaveDialog, ExportDialog, ShareDialog	New/open/save/export/share UX.
Preferences and utilities	PreferencesDialog, QuickHelpWindow, IconBrowser, ColorPalette	Configuration and supporting tools.

5. State Management Architecture

The active monolithic store is `store/projectStore.ts`. It defines `ProjectState` and creates `useProjectStore`. It holds domain state, UI state, transport state, persistence functions, recording actions, MIDI actions, and many feature-specific actions in one file.

State area	Representative fields/actions	Responsibilities
Transport	playing, recording, playhead, play, stop, seek, toggleRecording	Store-facing transport state and command dispatch.
Project	id, name, tempo, timeSignature, keySignature, initializeProject, loadProject	Project metadata and initialization.
Tracks/clips	tracks, clips, addTrack, updateTrack, deleteTrack, addClip, updateClip	Arrangement data model and editing operations.
Mixer/plugin state	plugins, sends, channelStripSettings, performances	Channel strip and plugin settings.
UI visibility	showLibrary, showMixer, showExportDialog, bottomPanel	Panel and modal state.
Recording	recordingStartTime, liveRecordingClips, startRecording, stopRecording	Audio/MIDI recording lifecycle coordination.
Persistence	saveProject, loadProject, openProject, local/global settings functions	Server and browser persistence operations.

State risk: There is also a `stores/` directory with slice-based stores and tests. The project currently has both `store/` and `stores/` patterns, so the architecture has a split state-system boundary.

6. Domain Model

The client-side domain model centers on Project, Track, Clip, Note, and Articulation. The server-side Prisma model mirrors similar concepts, but not perfectly.

Model	Client/server location	Key fields / relationship
Project	models/Project.ts, prisma Project	Metadata, tempo/signature/key, stateJson, tracks, buses.
Track	models/Track.ts, prisma Track	Type, volume, pan, mute/solo, plugins, sends, automation, clips.
Clip	models/Clip.ts, prisma Clip	Audio/MIDI region, start, duration, offset, fileId, notes.
Note	models/Clip.ts, prisma Note	Pitch, velocity, start, duration.
Automation	models/Track.ts, prisma Automation	Parameter lanes and automation points.
Plugin	models/Track.ts, prisma Plugin	Track insert/settings state.
Bus/Send	prisma Bus/Send plus client sends	Routing relationships and send levels.

7. Audio Engine Architecture

The current engine architecture is adapter-led. UI and store code import `audioEngine/audioEngine2` from `engine/AudioEngineAdapter.ts`. That adapter forwards operations into V2 modules.

Module	Responsibility	Architectural role
AudioEngineAdapter	Backward-compatible facade	Single import target for legacy store/components; routes to V2 modules.
audioContext.ts	AudioContext lifecycle, worklet loading, device access	Browser audio root and environment abstraction.
routingEngine.ts	Track/bus node graph, master output, gain/pan/sends	Signal routing and mixer mutation target.
scheduler.ts	Lookahead clip scheduler, worker tick loop, transport events	Timeline playback engine for audio clips.
recordingEngine.ts	V2 recording engine	Mic capture path, separate from newer audioRecording/recorder.ts path.
audioRecording/recorder.ts	DAW-grade recorder wrapper	Actual store-integrated recording path.
bufferCache.ts	AudioBuffer cache	Decoding/cache layer for playback.
bounceEngine.ts	Offline render/export	Export engine, but not fully wired to product ExportDialog.
SynthEngine, instruments/*	Synth, sampler, drum kit engines	Live MIDI/instrument sound generation.

7.1 Intended Audio Signal Flow

Audio clip buffer
-> AudioBufferSourceNode
-> RoutingEngine.trackNodes[clip.trackId].inputGain
-> preEffects[]
-> mainGain
-> postEffects[]
-> StereoPannerNode
-> send taps / buses
-> masterGain
-> outputNode
-> AudioContext.destination

Live MIDI note
-> AudioEngineAdapter.triggerNote(trackId, pitch, velocity, instrument)
-> SynthEngine or MultiSamplerEngine
-> track input/main gain
-> master chain
-> destination
Mic recording
-> MediaStreamAudioSourceNode
-> recorder worklet or ScriptProcessor fallback
-> RecordingBufferManager
-> RecordingClip
-> projectStore.addClipToTimeline

7.2 Current Audio Integration Gaps

Gap	Why it matters	Architectural consequence
Clip schema mismatch	Store Clip uses start; scheduler AudioClip uses startBeat.	The engine boundary needs a mapper/adaptor model, not direct casting.
Track creation passes partial objects	RoutingEngine expects volume, pan, effects, sends.	Track graph construction must use normalized AudioTrack defaults.
Multiple recording engines	engine/audioEngine/recordingEngine.ts and engine/audioRecording/recorder.ts coexist.	Recording ownership is unclear.
ExportDialog not wired to bounce engine	Store export action only logs/closes.	Product export architecture is incomplete.
Transport split	Store play logic and useAudioPlayer scheduler logic both initiate playback behavior.	One transport owner must be chosen.

8. DSP, WASM, Worklet, and Graph Compiler Architecture

The repository contains a separate next-generation DSP architecture under engine/audio, engine/dsp, rust, wasm, and public/worklets. This appears to target compiled graph execution and WASM-backed DSP nodes, but it is not the same as the current AudioEngineAdapter path.

Subsystem	Files	Purpose
DSP graph model	engine/audio/graph, engine/dsp/graph	Node/connection graphs, topological sorting, graph patches.
Compiler/runtime	engine/audio/compiler, engine/audio/runtime	Execution planning, latency resolving, runtime graph swapping, parameter writing.
WASM bridge	engine/audio/wasm, engine/dsp/runtime	WASM plugin loading, shared memory bridge, runtime context.
WASM kernels	wasm/dsp-core, rust/dsp-core, rust/plugins	EQ/compressor/DSP primitives and plugin experiments.
Worklets	public/worklets, public/recorder-worklet.js	AudioWorklet processors for synth/DSP/recording.
Plugin UI shells	components/plugins, ChannelEQ, Compressor, ChromaVerb, TapeDelay	Visual controls for DSP/plugin concepts.

Architectural boundary: The DSP/WASM graph stack should be treated as a future runtime lane unless it is explicitly booted and made the single mixer/playback path.

9. MIDI and Instrument Architecture

Subsystem	Files	Role
MIDI data model	engine/midi/types.ts, models/Clip.ts	MIDI notes, regions, editor state, quantize/humanize concepts.
Piano roll sync	components/PianoRoll.tsx, components/midi/*, engine/pianoRoll/projectSync.ts	Editor views synchronized to project clips/notes.
Live input	VirtualKeyboard, StepInputKeyboard, GlobalKeyHandler	User-triggered note input and step recording.
Instrument registry	engine/instruments/instrumentRegistry.ts, lib/libraryData.ts	Named sound metadata and preset mapping.
Synth engines	engine/SynthEngine.ts, engine/instruments/synthEngine.ts	Oscillator-based polyphonic synthesis.
Sampler engines	engine/instruments/samplerEngine.ts, multiSamplerEngine.ts	Sample maps and DecentSampler-like dspreset playback.
Drums	engine/instruments/drumMachine.ts	Synthesized drum kit presets and mappings.

Built-in local sample assets currently live under public/sound_sample with piano and guitar presets. Some loop data uses external URLs, which creates availability and licensing/deployment risk.

10. Timeline, Editing, and Rendering Architecture

Area	Files	Responsibility
Timeline UI	components/Timeline.tsx, TimelineCanvas, TimelineWithClipEditing	Region display, context menus, clip interactions.
Track headers	components/TrackList.tsx	Track selection, mute/solo/record, fader/pan headers, track menus.
Clip editing engine	engine/timeline/*	Crossfades, ghost clips, history, slip editing, clip tools, waveform cache.
Rendering foundation	engine/rendering/*, engine/gpu/*	Renderer scheduler, dirty regions, WebGL renderer scaffolds.
Editor core	engine/editor/*	Tools, snapping, coordinate system, selection manager.

11. Mixer and Automation Architecture

Mixer architecture spans visual React components, project store mutation, routing engine AudioNode mutation, and separate engine/mixer classes.

Layer	Files	Role
UI channel strips	components/Mixer.tsx, components/mixer/*	Faders, meters, sends, channel strip layout.
Store mutation	store/projectStore.ts updateTrack/updateProjectSettings	Persists volume, pan, mute, solo, settings.
Runtime routing	engine/audioEngine/routingEngine.ts	Mutates GainNode and StereoPannerNode values.
Mixer domain	engine/mixer/*	Bus, send, meter, mixer channel classes.
Automation UI	components/automation/*	Lane/point/curve editing.
Automation runtime	engine/automation/*	Parameter maps, schedulers, rendering, lookahead, commands.

12. Persistence, Storage, and Project Files

Persistence path	Files	Data stored	Notes
Server DB	app/api/project/save/route.ts, app/api/project/[id]/route.ts, prisma/schema.prisma	Users, projects, tracks, clips, notes, automation, plugins, buses, sends, stateJson	Uses PostgreSQL through Prisma.
Client store	store/projectStore.ts	Live app state	Primary in-memory source of truth.
Local project filesystem	engine/filesystem/projectManager.ts, projectSerializer.ts, indexedDBAdapter.ts	Serialized projects/assets/user settings	Appears to be an alternate/local project architecture.
Supabase storage	lib/supabase.ts, lib/storage.ts	Uploads/assets	Configured through NEXT_PUBLIC_SUPABASE_*
Browser file imports	components/Browsers.tsx, engine/audioImport.ts	Object URLs, decoded buffers, waveform peaks	Import path is browser-local unless persisted later.

13. Authentication and Account Architecture

Authentication is NextAuth.js credentials-based. The current authorize callback returns a demo user for any attempt. There is no real signup, password verification, account settings, or user-owned project isolation beyond userId fields being passed around.

Auth part	File	Current function
Provider	app/api/auth/[...nextauth]/route.ts	CredentialsProvider named Demo Account.
Session strategy	app/api/auth/[...nextauth]/route.ts	JWT session strategy.
Login UI	app/login/page.tsx	Single demo access button.
Dashboard guard	app/dashboard/page.tsx	Redirects unauthenticated users to /login.

14. Sound Library and Asset Architecture

Asset/source	Current contents	Architectural role
public/sound_sample/piano	108 WAV + Piano.dsmpreset	Local multisampled piano.
public/sound_sample/guitar	18 WAV + MG Soft Nylon Guitar preset	Local sampled guitar.
lib/libraryData.ts	Software instrument, synth, drum kit, keyboard preset names	Library UI catalog.
engine/soundLibrary/instruments.ts	Sound metadata	Instrument category/detail metadata.
components/LoopBrowser.tsx	Six hardcoded loop records; some remote URLs	Loop browser demo data.
magic-pro-modules	Experimental SF2/WAV sampler modules	Standalone prototype modules, not main runtime.

15. Deployment Architecture

Concern	Current architecture	Deployment implication
Framework	Next.js 14 App Router	Can deploy to Vercel/Node if build passes.
Database	Prisma PostgreSQL	Needs DATABASE_URL and migrations.
Auth	NextAuth	Needs NEXTAUTH_SECRET and production NEXTAUTH_URL.
Static audio assets	public/sound_sample	Bundled into deployment output; asset size should be reviewed.
AudioWorklet/WASM	public worklets and WASM build scripts	Needs correct static paths and cross-origin isolation if SharedArrayBuffer is required.
ESLint/tests	Scripts exist partially; lint prompts setup; no test script	CI quality gate is incomplete.
Generated output	.next is tracked	Repository hygiene issue; deployment output should not be source-controlled.

16. Key Architecture Risks

Risk	Severity	Architectural cause	Recommended fix
Dual/overlapping audio engines	High	Adapter, V2 engine, DSP runtime, legacy code coexist.	Choose one production audio runtime and isolate experiments.
Model mismatch across store and engine	High	Clip uses start; AudioClip uses startBeat; partial Track passed to full AudioTrack API.	Create explicit mapping layer with validated normalized engine DTOs.
Monolithic project store	High	ProjectStore holds transport, UI, recording, MIDI, persistence, editing, preferences.	Split store into slices behind a single typed facade.
Fake/export placeholder behavior	High	Export UI does not call real renderer.	Wire export UI to one offline render path and verify output.
Demo auth	High	Credentials authorize always returns demo user.	Implement real auth or remove account claims from v1.
No single source of truth for persistence	Medium	Prisma, localStorage, IndexedDB services, Supabase utilities coexist.	Define v1 persistence contract and remove unused paths from user flow.
Desktop-only interaction model	Medium	Large DAW UI with mouse interactions and dense dialogs.	Declare desktop-only v1 or build mobile-specific flow.

17. Recommended Target Architecture for V1

The v1 target should be a narrow vertical slice, not the full Logic-like surface. The architecture below minimizes launch risk while preserving the browser DAW premise.

V1 production path
React Studio UI
-> ProjectStore facade
-> TransportService
-> TrackService
-> ClipService
-> InstrumentService
-> PersistenceService

-> AudioRuntime facade		
-> AudioContextManager		
-> RoutingEngine		
-> Scheduler		
-> Recorder		
-> OfflineRenderer		
Rules:		
1. UI never talks to low-level AudioNodes directly.		
2. Store never passes raw UI models into the scheduler.		
3. Every Track/Clip is normalized before entering the engine.		
4. Recording, playback, preview, and export use the same routing model.		
5. Experiments live outside the default import path.		
V1 module	Owns	Should not own
ProjectStore facade	Serializable project state and action dispatch	AudioNode construction.
AudioRuntime	AudioContext, scheduling, routing, live notes, export	React state or UI visibility.
AssetService	Decode/cache/import/persist audio assets	Timeline editing rules.
InstrumentService	Default instruments, sound loading, note on/off	Project persistence format.
PersistenceService	Save/load project snapshots and assets	Engine scheduling.
Onboarding service	Templates, starter beat, first-run flow	Low-level DAW preferences.

18. Directory Map

Directory	Role
app/	Next.js routes, API endpoints, global CSS/providers.
components/	Main React DAW UI and dialogs.
components/midi, mixer, automation, filesystem, plugins	Feature-specific UI component groups.
store/	Current monolithic Zustand store and related stores.
stores/	Alternate/slice-based Zustand store structure.
engine/audioEngine/	V2 Web Audio engine modules.
engine/audio/	DSP graph/runtime/WASM-oriented experimental engine.
engine/audioRecording/	Store-integrated recording path.
engine/instruments/	Synth/sampler/drum instrument system.
engine/midi/	MIDI editor/runtime utilities.
engine/timeline/	Timeline editing/rendering utilities.
engine/filesystem/	IndexedDB/project serialization/import/export services.
prisma/	Database schema and seed.
public/	Static worklets and sound samples.
rust/, wasm/	Rust/WASM DSP core and plugin experiments.
legacy/, magic-pro-modules/	Older standalone prototypes and experimental modules.

19. Appendix: Main Data Flow Scenarios

19.1 Create a New Project

1. User lands at /dashboard after authentication.
2. ProjectChooser collects tempo, key signature, time signature, project format, surround/spatial settings.
3. dashboard/page.tsx calls initializeProject in store/projectStore.ts.
4. Router navigates to /project/new-project.
5. ProjectStudio mounts the studio layout and useAudioPlayer.

19.2 Add Track and Play a Note

1. NewTrackDialog calls addTrack with track type, color, icon, and record/input monitoring flags.
2. projectStore.addTrack calls audioEngine.createTrack and appends the track to state.
3. VirtualKeyboard or MIDI input calls projectStore.triggerNote.
4. projectStore resolves focused track and calls audioEngine.triggerNote.
5. AudioEngineAdapter creates or retrieves SynthEngine/MultiSamplerEngine and connects output to the track graph if available.

19.3 Import Audio and Playback

1. Browsers or import path creates a File object URL and calls addMediaFile/addClip.
2. useAudioPlayer sees audio clips with fileUrl and decodes them into audioBufferCache.
3. When playing becomes true, useAudioPlayer sends clips and tracks to advancedScheduler.startPlayback.
4. scheduler creates AudioBufferSourceNode objects and connects them to routingEngine track input nodes.
5. routingEngine sends track signal through gain/pan/master/output to AudioContext.destination.

19.4 Save and Load

1. Project store serializes tracks, clips, global tracks, settings, alternatives, global settings, and environment.
2. saveProject posts to /api/project/save.
3. API route upserts Project and recreates nested Track/Clip/Note rows.
4. loadProject fetches /api/project/[id], reconstructs project state, and applies tempo/audio format to the engine adapter.

20. Appendix: Launch-Oriented Architecture Checklist

Area	Required v1 architecture decision
Playback	One scheduler owner, one clip DTO, one track graph lifecycle.
Mixer	All audio, preview, recording monitoring, MIDI instruments, and export must pass through the same routing graph.
Sounds	At least one default drum kit and one default instrument must load with no user upload.
Onboarding	First run should create a starter project or guided first beat path.
Persistence	Define whether v1 source of truth is Prisma, IndexedDB, or both; document asset handling.
Export/share	Render actual WAV and share/download real files, not JSON placeholders.
Auth	Either real accounts or explicit no-account local mode; demo auth cannot be v1 production auth.
Deployment	Build must pass, migrations must exist, generated files must be untracked.